

README

Part 18: RAG 全链路 — 手写五件套、上下文增强与"何时不该用 RAG"

- > 应用线 A1 的第一站。算法主线（Part 1-17）教会模型"会算"，应用线教
- > "会用"：RAG（检索增强生成）是 LLM 应用工程师赛道的第一块硬通货——
- > 面试指南簇 D+E（RAG/Agent, ★★★★★）的核心。本部分不调 LangChain、不配
- > 向量数据库，在本仓库 8 篇真实 Markdown（固定快照 [data/part18_corpus/](#)）上从零手写工业 RAG
- > 管线的每一层：递归分块 → 稠密嵌入（Qwen3-Embedding，官方 last-token
- > pooling）→ 手写 BM25 → RRF 混合 → cross-encoder 重排 → 0.5B 生成带引用回答
- > → 手写 RAGAS 评测。所有模型缺失时自动降级（hashing trick / 抽取式 /
- > 关键词裁判），脚本永不崩。
- > 与算法主线的互文：检索的"近似哲学"接 [Part 13 数据工程](../Part13_data_engineering/tutorial/README.md)，
- > 生成的服务化接 [Part 14 推理部署](../Part14_inference_vllm/tutorial/README.md)，
- > "检索变成工具"预告 Part 19（Agent）。

学习目标

完成本部分后，你将能够：

- 手写 RAG 五件套并用 recall@k 消融表量化每件套的贡献（本机实测单路 0.58/0.60 → 混合 0.85 → +重排 0.92）
- 复现 Anthropic contextual retrieval 四阶梯实验，并对照官方数字（失败率 5.7%→1.9%）解释本机量级差异——包括"格式噪声与增益同量级"这一反直觉实测
- 手写 RAGAS 的 faithfulness / context precision（0.5B 当裁判），并演示与防御"评测器噪声"
- 决策 RAG vs 微调 vs 长上下文（LaRA 无银弹 / Self-Route 的成本路由 / 装得下就直塞的 context engineering 共识）
- 设计 模型缺失时的降级路径，让整条管线在任何环境 rc=0

章节导航

	内容	对应脚本
	递归分块/BM25 推导/RRF/cross-encoder/last-token pooling/降级路径；recall@5 四级消融实测	[<code>'01_minimal_rag.py'</code>](../rag.md)
	contextual retrieval 复刻 + late chunking；GraphRAG/HippoRAG 2/RAPTOR（认知）；RAGAS 四指标手写与评测器噪声	[<code>'02_contextual_retrieval.py'</code>](../rag.md)

前置知识

- 必须掌握：
- [Part 6 Transformer](../Part6_transformer/tutorial/README.md)——嵌入模型与

cross-encoder 都是 Transformer 编码器；cosine = 归一化点积

- [Part 8 SFT](../Part8_post_training/tutorial/README.md)——instruct 模型与 chat template（生成/裁判环节全靠它）
- 建议掌握：
- [Part 13 数据工程](../Part13_data_engineering/tutorial/01_dedup_from_scratch.md)——"精确算不动就设计可算的近似"的检索哲学（LSH ↔ BM25 互文）；语料就来自本仓库 docs/ 的固定快照（data/part18_corpus/, 8 篇，经 Part 13 思想挑选）
- 可选：
- [Part 14 推理部署](../Part14_inference_vllm/tutorial/README.md)——生产 RAG 的生成侧要架在 vLLM/SGLang 上；长上下文的 KV cache 成本结构是 02 章"何时不该用 RAG"的物理基础

在 LLM 链路中的位置

```
Part 13（数据工程：语料从哪来）——┐
Part 6/8（模型：会读会写）———┐→ 【本部分: 给模型外挂一个可检索的记忆】
Part 14（推理部署：跑得快）——┘┐
Part 19（Agent: 检索变成模型手里的工具，待开）┘
```

RAG 是应用线的地基：Agent 的"查资料"动作、长上下文应用的"知识底座"、面试的"系统设计题"（面试指南跨方向高频考点 TOP15 第 13 条），全都从这条管线讲起。

环境

```
`bash
```

模型（首次运行自动从 HF
缓存读取；缺失时自动降级并打印下载指引）

Qwen/Qwen3-Embedding-0.6B 检索嵌入（fp32）

Qwen/Qwen2.5-0.5B-Instruct 生成/上下文生成/裁判（fp16）

BAAI/bge-reranker-v2-m3 cross-encoder 重排（fp32）

```
cd courses/Part18_rag/scripts
```

```
CUDA_VISIBLE_DEVICES=0 python 01_minimal_rag.py # ~13-15s（RTX 4090，共享 GPU 有波动）
```

```
CUDA_VISIBLE_DEVICES=0 python 02_contextual_retrieval.py # ~50-55s
```

```
CUDA_VISIBLE_DEVICES=0 python 03_rag_eval.py # ~17s
```

体验降级路径（零模型、纯 CPU，约
3s，验证"永不崩"设计）

```
RAG18_FORCE_FALLBACK=1 python 01_minimal_rag.py
```

- GPU 与其他任务共享时先 `nvidia-smi` 挑空卡；语料仅 8 篇 md，纯 CPU 也可接受（0.5B 生成约慢 10 倍，嵌入/重排更慢，但全部有降级路径兜底）
- ragas 为可选依赖（未装时脚本 03 打印 `uv pip install --python .venv ragas` 指引后跳过该段，rc=0）

学习地图

五件套（分块→嵌入→BM25→RRF→重排→生成）← 点：每一件都能单独消融
 ↓ recall@5 消融：0.58/0.60 → 0.85 → 0.92（01 章实测）
 上下文增强（contextual retrieval / late chunking / 结构化前缀）← 线：chunk 失语境问题
 ↓ 格式噪声与增益同量级（02 章实验二实测 ±0.08）
 评测（RAGAS 四指标手写 + 评测器噪声）← 线：答案质量 ≠ 检索质量
 ↓
 边界决策（RAG vs 微调 vs 长上下文）← 面：什么时候根本不该用 RAG
 ↓ Part 19：检索变成工具，Agentic RAG

课后作业

[Assignment 18](../..../assignments/assignment_18/)——手写
`recursive_chunk` / `bm25_scores` / `rrf_fuse` / `faithfulness` 四件核心
 （与课程脚本同名同签名），题 5 网格搜 hybrid 权重画 recall- α 曲线。

相关资源

- Lewis et al. 2020 《RAG for Knowledge-Intensive NLP Tasks》（arXiv [2005.11401](https://arxiv.org/abs/2005.11401)）
- Anthropic 《Introducing Contextual Retrieval》([engineering blog](https://www.anthropic.com/engineering/contextual-retrieval)) · Late Chunking（arXiv [2409.04701](https://arxiv.org/abs/2409.04701)）
- Agentic RAG 综述（[2501.09136](https://arxiv.org/abs/2501.09136)） · GraphRAG（[2404.16130](https://arxiv.org/abs/2404.16130)） · HippoRAG 2（[2502.14802](https://arxiv.org/abs/2502.14802)） · RAPTOR（[2401.18059](https://arxiv.org/abs/2401.18059)）
- LaRA（[2502.09977](https://arxiv.org/abs/2502.09977)） · Self-Route（[2407.16833](https://arxiv.org/abs/2407.16833)） · MTEB 维护性研究（[2506.21182](https://arxiv.org/abs/2506.21182)） · [RTEB](https://github.com/NovaSearch-Team/RTEB)
- [RAGAS](https://github.com/explodinggradients/ragas) · [Qwen3-Embedding 模型卡](https://huggingface.co/Qwen/Qwen3-Embedding-0.6B) · [BAAI/bge-reranker-v2-m3](https://huggingface.co/BAAI/bge-reranker-v2-m3)

[← 上一部分：Part 17 Agentic RL](../..../Part17_agentic_rl/tutorial/README.md) |
 [返回课程总览](../..../README.md)

下一站 Part 19（Agent，待开）：本部分的检索管线将成为模型的工具——

什么时候查、查什么、查完够不够，都交给模型决策（Agentic RAG），训练方法接 [Part 17 的多轮轨迹 RL](../Part17_agentic_rl/tutorial/01_from_single_turn_to_agent.md)。

01_naive_to_hybrid

01 — 从朴素 RAG 到混合检索：手写五件套

- > Part 6 你手写了注意力，Part 8 你微调了 instruct 模型——但模型的知识仍被锁在
- > 参数里（闭卷考试）。本章在本仓库 8 篇真实 Markdown（固定快照
- > [data/part18_corpus/](#)，取自 docs/）上，不借助任何
- > RAG 框架，手写最小而五脏俱全的检索增强管线五件套：
- > 递归分块 → 嵌入 → BM25 → RRF 混合 → cross-encoder 重排，最后让 0.5B 模型
- > 带着证据回答。跑 [scripts/01_minimal_rag.py](../scripts/01_minimal_rag.py)
- > （RTX 4090 实测 13-15 秒；模型缺失时自动降级，脚本永不崩）。

学习目标

完成本章后，你将能够：

- 手写 RAG 五件套：递归分块 / 稠密嵌入 / BM25 / RRF 融合 / cross-encoder 重排
- 推导 BM25 的 TF 饱和与文档长度归一项、RRF 的倒数排名公式
- 解释 dense 与 BM25 各自的盲区，以及"混合 + 重排"为什么能逐级抬升 recall
- 配置 Qwen3-Embedding 的官方用法（last-token pooling、查询侧指令前缀）
- 设计可复现的降级路径（hashing trick 嵌入、跳过重排），让管线在没有 GPU / 没有模型的环境里依然 rc=0

前置知识

必须掌握：

- [Part 6 Transformer](../Part6_transformer/tutorial/README.md)——嵌入模型和 cross-encoder 本质都是 Transformer 编码器；cosine 相似度就是点积（Part 6 注意力里 QK^T 的归一化版）
- [Part 8 SFT](../Part8_post_training/tutorial/README.md)——instruct 模型与 chat template（生成环节用 Qwen2.5-0.5B-Instruct 的对话模板拼 prompt）

建议掌握：

- [Part 13 数据工程](../Part13_data_engineering/tutorial/01_dedup_from_scratch.md)——"精确算不动就设计可算的近似"的哲学一脉相承：LSH 用分带哈希近似 Jaccard，BM25 用 TF 饱和近似"词项重要性"；本仓库语料的清洗/去重也发生在这一站
- [Part 8 量化与服务](../Part8_post_training/tutorial/README.md)——fp16/fp32 的显存权衡（本章嵌入模型用 fp32、生成模型用 fp16 的原因）

可选：

- [Part 14 推理部署](../Part14_inference_vllm/tutorial/README.md)——生产级 RAG 的生成侧要架在 vLLM/SGLang 上，检索侧只是它前面的一个模块

理论背景

问题引入：为什么需要 RAG？

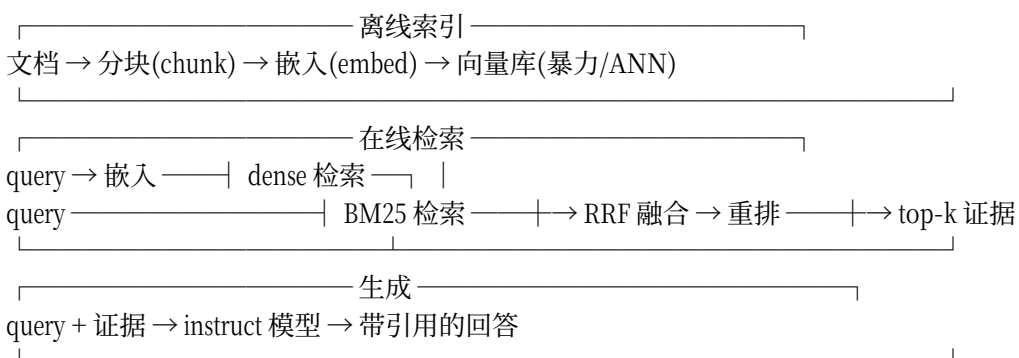
没有检索增强之前，让 LLM 回答知识型问题有三个绕不开的痛点：

1. 知识截止：参数里的知识冻结在训练截止日，问“我们仓库的路线图”必然瞎编
2. 私有数据不可见：内部文档、数据库、本仓库的 8 篇 md 从未进过训练集
3. 幻觉无追溯：模型给出的“事实”没有出处，无法审计

> 类比：微调是“让学生把教材背下来再去考试”（贵、慢、背不动新教材）；
> RAG 是“开卷考试”——先去书架（检索）翻出相关页（chunk），再照着答题（生成）。
> 背书擅长“风格与能力”，翻书擅长“事实与出处”，两者不冲突（后面 02 章会讲怎么选）。

RAG 的最初形态（Lewis et al. 2020, arXiv [2005.11401](https://arxiv.org/abs/2005.11401)）

就是把一个可微检索器接进生成器。今天工业界的标配流水线长这样：



数学推导

① BM25：从 TF-IDF 到“饱和 + 长度归一”

直觉：一个词在一篇文档里出现 10 次，不等于比出现 1 次“重要 10 倍”；
一篇长文档堆词的机会天然更多，不 penalize 长度就会偏向长文。

推导：

Step 1: TF-IDF 起点

$\text{score}(t, d) = \text{tf}(t, d) \cdot \text{IDF}(t)$, $\text{IDF}(t) = \log(N / \text{df}(t))$

问题 1：tf 线性增长 → 长文刷分；问题 2：IDF 在 $\text{df} \rightarrow N$ 时趋于 0 甚至为负

Step 2: TF 饱和（乘一个渐近线为 (k_1+1) 的因子）

tf 部分改为 $\text{tf} \cdot (k_1+1) / (\text{tf} + k_1)$

→ $\text{tf} \rightarrow \infty$ 时趋于 (k_1+1) ； k_1 控制饱和速度（经验默认 1.2）

Step 3: 文档长度归一（BM25 最终形态）

$\text{tf} \cdot (k_1+1) / (\text{tf} + k_1 \cdot (1 - b + b \cdot |d|/\text{avgdl}))$

→ $|d| = \text{avgdl}$ 时因子为 1（不奖不罚）； b 控制归一强度（经验默认 0.75， $b=0$ 完全不看长度， $b=1$ 完全按长度缩放）

Step 4: 平滑 IDF（避免负值，本课程实现采用）

$\text{IDF}(t) = \ln(1 + (N - \text{df} + 0.5) / (\text{df} + 0.5))$ —— 恒正

最终： $\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \text{tf} \cdot (k_1 + 1) / (\text{tf} + k_1 \cdot (1 - b + b \cdot |d| / \text{avgdl}))$

- > 关键概念：BM25 是"词法检索"——只看字面 token 是否匹配，完全不懂
- > "组内相对策略梯度"和"GRPO"是一回事。这正是它的盲区，也是 dense 检索的用武之地。
- > 与 Part 13 的互文：LSH 用分带哈希把 $O(N^2)$ 的 Jaccard 比较变成近似可算；
- > BM25 用 TF 饱和 + 长度归一把"词项重要性"变成可算的打分。工程检索的智慧从来
- > 不是"算得更准"，而是"把不可算的目标准则改造成可算的代理"。

② RRF：只融合名次，不融合分值

dense 给的是 cosine $([-1, 1])$ ，BM25 给的是无界正分——两把尺子量出的数字不可直接加。加权融合要先做尺度标定（min-max？z-score？），标定错了就全错。

RRF（Reciprocal Rank Fusion）的答案：丢掉分值，只看名次。

$\text{score}(\text{item}) = \sum_{\{\text{每个榜单}\}} 1 / (k + \text{rank}(\text{item}))$ ，rank 从 1 起， $k = 60$ （论文默认）

直觉：第 1 名得 $1/61$ ，第 2 名得 $1/62$ ……名次差 1 的得分差被 k 压平，

于是一个"两个榜单都进前 10"的文档轻松赢过"单榜第 1"。

极限： $k \rightarrow \infty$ 时 $1/(k + \text{rank}) \approx (1 - \text{rank}/k)/k \rightarrow$ 退化为"入选榜单数优先、名次和次之"的计数排序（作业题 3 会让你用测试验证这个性质）。

- > 关键概念：RRF 天然免尺度标定、免调参——这是它取代加权混合成为工业
- > 默认的原因。但"免调参"不等于"最优"：权重网格搜索仍能挤出最后几个点
- > （作业题 5 就是这个实验）。

③ 稠密检索：cosine 与 last-token pooling

嵌入模型把文本映射到单位球面上的向量，相关文本夹角小：

$\cos(q, d) = q \cdot d / (\|q\| \cdot \|d\|)$ 实现上先 L2 归一化 $\rightarrow$ cosine 退化为一次矩阵乘
 $\text{sims} = \text{chunk_mat} @ \text{q_vec} \# (N, D) @ (D,) \rightarrow (N,)$

Qwen3-Embedding 是因果（decoder-only）嵌入模型，官方用法是取

最后一个有效 token 的隐状态做

pooling（[官方模型卡](https://huggingface.co/Qwen/Qwen3-Embedding-0.6B)），

且查询侧要拼任务指令前缀：

```
python
```

```
QUERY_INSTR = ('Instruct: Given a web search query, retrieve relevant passages '
```

```
'that answer the query\nQuery: ')
```

文档侧不加前缀；池化用 last_token_pool（左填充直接取末列，右填充逐样本取最后有效位）

- > △ 拿 BERT 式 mean-pooling 去用 Qwen3-Embedding 是最常见的"静默劣化"——
- > 不报错、不掉零，就是召回悄悄变差（见本章陷阱 3）。

④ Cross-encoder 重排：为什么贵一个量级却还值得

bi-encoder（嵌入检索）把 query 和文档各自编码再比对——可以离线算、可以 ANN，但两个向量从没“见过面”。cross-encoder 把 (query, 文档) 拼成一对一起过模型，注意力可以在每个 token 层面互相对质——精度高一个量级，算力也贵一个量级（无法离线、无法 ANN）。所以工业管线的定式是：**便宜的 bi-encoder 从百万里捞 top-10/20，昂贵的 cross-encoder 只对这几十个精排**。

> 类比：bi-encoder 是“两人各写一份简历再比对关键词”，cross-encoder 是“当面逐句对质”。

历史脉络

- 2020：Lewis et al. 提出 RAG (arXiv [2005.11401](https://arxiv.org/abs/2005.11401))，DPR ([2004.04906](https://arxiv.org/abs/2004.04906)) 确立双塔检索
- 1990s→今：BM25 (Okapi, 1994) 从图书馆检索活到今天的搜索引擎默认基线——五件套里最老的零件反而是最扛造的
- 2023：cross-encoder 重排 + RRF 混合成为开源检索栈标配 (Weaviate/Elastic 同年内置 RRF)
- 2024-25：contextual retrieval (Anthropic)、late chunking (Jina) 修补“chunk 失去上下文”的结构缺陷 (→ [02 章](02_advanced_rag.md))
- 现在：Agentic RAG 把“检索几轮、检索什么”也交给模型决策 (→ Part 19)

代码实现

数据流与形状追踪

```
part18_corpus/ 8 篇 md (共 ~86k 字符)
↓ recursive_chunk(size=512, overlap=64) 字符级贪心装箱
chunks: list[str] × 238
↓ Qwen3-Embedding-0.6B (fp32) + last-token pooling + L2 归一
chunk_mat: (238, 1024) ← 降级路径: hash_embed → (238, 256)
↓ q_vec: (1024,) (查询侧带 Instruct 前缀)
dense 检索: sims = chunk_mat @ q_vec → (238,) → top-10 名单
BM25 检索: bm25_scores(query, chunks) → list[float] × 238 → top-10 名单
↓ rrf_fuse(dense_top10, bm25_top10, k=60) 只融合名次
hybrid_top: list[int] × 10
↓ bge-reranker-v2-m3: (query, chunk) 成对打分 → logits (10,)
rerank_top: list[int] × 5 → top-3 作为生成证据
↓ Qwen2.5-0.5B-Instruct + chat template (证据编号 [1][2][3])
answer: str (句末标 [编号])
```

逐行解释

五件套之一：递归分块


```

python
def recursive_chunk(text, size=512, overlap=64):
    max_atom = size - overlap - 2 # 给 overlap 前缀 + 连接空格留位

    def split_atoms(s, seps): # 优先大分隔符，切不动就下钻小分隔符
        if len(s) <= max_atom:
            return [s]
        if not seps: # '\n\n'→'\n'→'。'→' ' 都切不动 → 硬切
            return [s[i + max_atom] for i in range(0, len(s), max_atom)]
        sep, rest = seps[0], seps[1:]
        pieces = []
        for part in s.split(sep):
            pieces.extend(split_atoms(part, rest))
        return pieces

    atoms = [a for a in split_atoms(text.strip(), ['\n\n', '\n', '。', ' ']) if a.strip()]
    chunks, cur = [], ''
    for atom in atoms:
        if cur and len(cur) + len(atom) + 1 > size: # 装不下 → 结算
            chunks.append(cur)
            cur = cur[-overlap:] # 上下文桥：跨块语义靠这 64 个字符续命
        cur = atom if not cur else cur + ' ' + atom
    if cur.strip():
        chunks.append(cur)
    return chunks

```

- 为什么递归：优先在段落边界切（语义完整），段落本身超长才下钻到句子、空格——这是 LangChain `RecursiveCharacterTextSplitter` 的同款思想
- 为什么 overlap：一句话恰好被切在边界上时，64 字符重叠保证它的头或尾至少完整出现在一个 chunk 里
- 三条可测试的不变量（作业题 1 就是测它们）： $\text{len}(\text{chunk}) \leq \text{size}$ ；相邻 chunk 首尾重叠恰为 `overlap`；去掉重叠段拼接后不丢任何非空白字符

五件套之二：手写 BM25

```

python
def bm25_scores(query, chunks, k1=1.2, b=0.75):
    n = len(chunks)
    doc_toks = [_tokens(c) for c in chunks] # 中英混合：英文词 + 中文二元
    avgdl = sum(len(d) for d in doc_toks) / n # 平均文档长度
    df = {}
    for dt in doc_toks: # 文档频率 df（含 df 的词 IDF 低）
        for term in set(dt):
            df[term] = df.get(term, 0) + 1
    scores = []
    for dt in doc_toks:
        tf = {}
        for term in dt:
            tf[term] = tf.get(term, 0) + 1
        s = 0.0
        for qt in _tokens(query): # 查询词按出现次数累加

```



```

if qt not in tf:
    continue
idf = math.log(1 + (n - df[qt] + 0.5) / (df[qt] + 0.5))
s += idf tf[qt] (k1 + 1) / (
    tf[qt] + k1 (1 - b + b len(dt) / avgdl))
scores.append(s)
return scores
`

```

中文没有空格，`_tokens` 对中文取单字 + 相邻二字 bigram——这是 BM25 处理中文的经典做法（作业里我们把它作为已提供的辅助函数，你专注 IDF/TF 主干）。

五件套之三：RRF 融合

```

`python
def rrf_fuse(list_a, list_b, k=60):
    scores, order = {}, {}
    for lst in (list_a, list_b):
        for rank, item in enumerate(lst, start=1):
            scores[item] = scores.get(item, 0.0) + 1.0 / (k + rank)
            order.setdefault(item, len(order)) # 记录首次出现序，用于并列稳定
    return sorted(scores, key=lambda it: (-scores[it], order[it]))
`

```

两行核心，一行防御（并列名次按出现顺序稳定输出——作业题 3 测这个）。

五件套之四：嵌入（含降级路径）

```

`python
def hash_embed(text, dim=256):
    """降级嵌入：hashing trick（特征哈希）——确定性、零模型依赖。"""
    vec = torch.zeros(dim)
    toks = _tokens(text)
    grams = toks + [toks[i] + toks[i + 1] for i in range(len(toks) - 1)]
    for g in grams:
        h = hashlib.md5(g.encode('utf-8')).digest() # md5：不受 PYTHONHASHSEED 影响
        idx = int.from_bytes(h[:4], 'little') % dim # 哈希到桶
        sign = 1.0 if h[4] % 2 == 0 else -1.0 # 随机符号抵消碰撞偏置
        vec[idx] += sign
    return F.normalize(vec, dim=0)
`

```

- > hashing trick 的价值：① 让脚本在“模型没下载/没有 GPU”时依然完整跑通
- > （本章实测：dense 列 recall 从 0.58 掉到 0.20——嵌入模型的贡献直接可视化，降级日志 /tmp 可复现）；
- > ② 它本身是工业老技术（Wowpal Wabbit 时代的大规模类别特征编码），语义为零、
- > 字面可用，正好用来体会“嵌入到底给了你什么”。

向量检索刻意用暴力广播 cosine：`sims = chunk_mat @ q_vec`，一次矩阵乘算完 238 个 chunk。注释里写明：不手写 ANN（HNSW/IVF）——百万级语料换 FAISS 的 `IndexFlatIP` 起步，思想不变，索引结构才是新东西。

五件套之五：重排与生成

```
`python
```

重排：把 (query, chunk) 成对喂给 bge-reranker-v2-m3, logit 越大越相关

```
logits = model(**tok(pairs, ...)).logits.squeeze(-1) # (B,)
order = torch.argsort(logits, descending=True)
```

生成：证据编号 [1][2][3] 喂给
0.5B-Instruct（小模型对数字编号的遵从度
远高于长格式引用），贪心解码保证可复现

```
`
```

调试展示：三个真实错误

错误 1：模型未下载直接崩

症状：

```
`
```

```
OSError: We couldn't connect to 'https://huggingface.co/Qwen/Qwen3-Embedding-0.6B'
```

```
`
```

原因：AutoModel.from_pretrained 在缓存缺失 + 无网络时抛异常，整条管线死在第一屏。

解法：load_embedder() 用 try/except 包住加载，失败返回 None，调用方走 hash_embed 降级并打印大写警告 + huggingface-cli download 指引。重排器/生成器同理（跳过 / 抽取式降级）。教程级脚本的铁律：降级路径不崩、rc=0。

错误 2：路径依赖当前目录

症状：从仓库根目录跑 python courses/Part18_rag/scripts/01_minimal_rag.py 正常，从别的目录跑报 FileNotFoundError: .../part18_corpus/...。

原因：相对路径按 cwd 解析，而运行目录不保证。

解法：一律 os.path.dirname(os.path.abspath(__file__)) 起算（本仓库脚本规范，Part 13 起就写在 scripts-guide 里）。

错误 3：Qwen3-Embedding 用了 mean-pooling

症状：不报错，但 dense 检索 recall 明显偏低、top-1 常是无关文档。

原因：因果嵌入模型的有效语义集中在最后一个 token（它看完了全文）；对中间 token 做 mean 会稀释掉“读完全文后的总结态”。

解法：照官方实现 last_token_pool（注意左右填充分支），查询侧拼 Instruct: ... Query: 前缀。

实测输出

> 环境标注：RTX 4090 (24GB), torch 2.6.0+cu124, transformers 4.57.6 ;
> Qwen3-Embedding-0.6B fp32, Qwen2.5-0.5B-Instruct fp16, bge-reranker-v2-m3 fp32 ;
> 语料 = data/part18_corpus/ 固定快照 (8 篇 md → 238 个 chunk, min/mean/max = 93/420/512 字符)
> ——快照固定, 教程数字可复现 (脚本缺快照时自动退回 docs/, 此时数字随 docs/ 更新漂移) ;
> 总耗时 13-15s (实测 13.2-13.8s ; 共享 GPU 上多次运行有波动) 。

[Step 3] 检索对比：dense / BM25 / hybrid(RRF k=60) / +rerank, 指标 recall@5

Q1: GRPO 出自哪篇论文？课程里用哪个框架跑它的实战？ ← 语义型查询

ground truth: 4 个相关 chunk

dense recall=0.75 bm25 recall=0.00 hybrid recall=0.75 +rr recall=0.75

[debug] Q1 与 dense 第一名的 cosine = 0.6566 (查询侧带官方 Instruct 前缀)

Q2: 面试要能默写的手写代码清单叫什么？ ← 词典型查询

ground truth: 14 个相关 chunk

dense recall=0.40 bm25 recall=1.00 hybrid recall=0.80 +rr recall=1.00

Q3: 预训练数据去重为什么能提升模型质量？ ← 综合型查询

ground truth: 9 个相关 chunk

dense recall=0.60 bm25 recall=0.80 hybrid recall=1.00 +rr recall=1.00

=====

query	dense	bm25	hybrid	+rerank
Q1	0.75	0.00	0.75	0.75
Q2	0.40	1.00	0.80	1.00
Q3	0.60	0.80	1.00	1.00
mean	0.58	0.60	0.85	0.92

=====

逐行解读这张表 (这是本章最重要的 30 秒)：

1. Q1 (dense 赢, BM25 零分)：查询问"出自哪篇论文/哪个框架", BM25 被 "论文、出自、框架"这些中文常用 bigram 淹没——paper_reading_guide 里几十个 chunk 都"谈论文"；而嵌入模型懂"组内相对策略梯度 ≈ GRPO"的语义近邻

2. Q2 (BM25 满分, dense 抓瞎)："清单叫什么"是纯字面问题, 稀有 token TOP8 的 IDF 一击命中；而"清单/默写/叫什么"在嵌入空间里离每个候选 chunk 都不远不近——没有语义近邻可用

3. hybrid (0.85) > 两个单路 (0.58/0.60)：RRF 把两份互补的名单叠起来——单路的盲区互相补位。注意 Q2 hybrid 反而比 BM25 低 (1.00→0.80)：

融合不是免费的, 弱路会把强路的好名次挤出去一点点

4. +rerank (0.92)：cross-encoder 在 10 个候选里精排, 把被融合挤掉的

相关 chunk (Q2) 捞回 top-5。重排只重排不召回——它救不了不在候选池里的文档

生成环节 (同一管线的 top-3 证据喂 Qwen2.5-0.5B-Instruct, 贪心解码)：

Q1: GRPO 出自哪篇论文？课程里用哪个框架跑它的实战？

证据: ['paper_reading_guide.md#c21', 'course_roadmap_v3.md#c42', 'course_roadmap_v3.md#c43']

回答: GRPO 出自《DeepSeekMath》这篇论文, ……课程中提到的实战包括：

1. 快速上手：0.5B GRPO 实战 (CLI 实操, Docker → 双卡)。……

Q2: 面试要能默写的手写代码清单叫什么？

证据: ['llm_interview_guide.md#c19', 'llm_interview_guide.md#c20', 'course_roadmap_v3.md#c23']

回答: 手写代码清单叫"TOP8"。

- > 0.5B 模型对"句末标 [编号]"的遵从不稳定（Q1 答对了内容但没带编号）——
- > 证据机制在 prompt 里、抽取式降级里都有，但小模型的指令遵从是概率性的。
- > 这正是 03 章要用"裁判模型"量化答案质量的原因。

降级路径实测（RAG18_FORCE_FALLBACK=1，同一脚本、零模型）：

△ RAG18_FORCE_FALLBACK=1 —— 强制使用 hashing trick 降级嵌入
chunk 矩阵: (238, 256)，耗时 0.4s，设备 cpu
query | dense | bm25 | hybrid | +rerank
mean | 0.20 | 0.60 | 0.40 | 0.40 ← dense 列从 0.58 掉到 0.20
回答走抽取式降级（挑含查询关键词的句子 + [k:来源] 引用）

hashing trick 只有字面碰撞信号：dense 列掉到 0.20，**这 0.38 的差（0.58→0.20）就是嵌入模型买到的东西**。降级不只是"不崩"，它本身就是一次消融实验。

工程实践

性能分析

操作	时间复杂度	本机实测（238 chunk）	百万级语料时
递归分块	$O(\text{字符数})$	<0.1s	分钟级（可并行）
嵌入（0.6B fp32）	$O(N \cdot L \cdot d^2)$	2.8s（batch=16）	GPU 小时级，一次离线
暴力 cosine	$O(N \cdot d)$	<0.01s（一次矩阵乘）	不可行 → ANN（FAISS/HNSW）
BM25	$O(N \cdot \text{平均词数})$	<0.1s	倒排索引毫秒级
cross-encoder 重排	$O(C \cdot L \cdot d^2)$ ，C=候选数	~0.3s / 10 候选	只重排 top-10/20
0.5B 生成	$O(\text{输出长度})$	~1s / 220 token	vLLM 批量（→ Part 14）

- > 检索侧的工业分水岭就在"暴力 cosine → ANN"这一行：N 小于几万时暴力
- > 反而最快且无损（ANN 是有损的）；不要为了"看起来专业"提前上 FAISS。

常见陷阱

陷阱 1：chunk 切得太碎，上下文丢失

症状：检索指标（recall@k）很好，但生成答案"对不上问题"——检索回来的是半句话/半张表，模型看到的关键词全在，语义链条断了。

原因：chunk 是检索单位也是生成证据单位；切得太碎，证据本身就是残句。

（本部分实测：同一组查询在 size=180 下 plain recall@20 均值从 0.65 掉到 0.53（单变量探针，口径与 02 章主实验略有差异；969 个碎 chunk；Q2 0.43→0.21 最惨），contextual 前缀也救不全——动手练习 3 可复现。）

解法：

python

chunk_size=64：关键词在、语义断

常用起点 256-1024 字符 + overlap 10%-20%，再按【下游任务指标】(不是检索指标) 调

`chunks = recursive_chunk(text, size=512, overlap=64)`

更好的证据单位 $\neq$ 更好的检索单位——生产系统常用"小块检索、大块返回"
(sentence-window / parent-child retrieval)。

陷阱 2：hybrid 权重拍脑袋，不网格搜

症状：加了 BM25 混合，指标反而降（本章 Q2：BM25 单路 1.00 $\rightarrow$ hybrid 0.80）。
原因：两路质量不对称时，对称融合（RRF 对两路平等）会稀释强路；加权融合的权重 α 更是超参数，拍脑袋必错。
解法：固定评测集后网格搜 α （dense 权重 0 $\rightarrow$ 1 步长 0.1），画 recall- α 曲线取最优——这正是作业 题 5 `hybrid_weight_sweep` 要做的事。RRF 的 $k=60$ 只是"免调参的稳健默认"，不是最优解。

陷阱 3：迷信嵌入模型榜单（MTEB）

症状：按 MTEB 榜换了"第一名"模型，自己语料上 recall 反而下降。
原因：榜单数字依赖特定的数据集组合、版本与环境——MTEB 维护性研究（arXiv [2506.21182](https://arxiv.org/abs/2506.21182)）的核心工作就是把基准的可复现性当工程问题对待（CI、数据集完整性检查、自动测试），说明榜单排名本身就是需要被"维护"的易碎品；真实任务上还应参考 [RTEB](https://github.com/NovaSearch-Team/RTEB)（Real-world Text Embedding Benchmark）这类贴近生产的评测。
解法：选型流程 = 榜单粗筛 $\rightarrow$ 自己的语料 + 自己的查询集上复测 $\rightarrow$ A/B 上线。任何"通用第一名"都要过你自己的这一关。

陷阱 4：查询侧忘了加指令前缀（或文档侧错加）

症状：换上 Qwen3-Embedding 后 recall 不如老模型。
原因：官方用法要求查询侧拼 `Instruct: ... Query:` 任务指令、文档侧不拼；两侧写反或漏写都会静默掉点。
解法：照抄官方 snippet（`embed_texts(..., is_query=True)` 分支），换模型时先跑官方 sanity check 再接管线。

最佳实践

1. 先跑通朴素版，再逐级加件：五件套每一件都能单独消融（本章表格就是 4 级消融）——不做消融的 RAG 优化等于蒙眼调参
2. 检索指标与生成指标分开看：recall@k 高不代表答案好（证据太碎）、答案好也不代表检索对（模型自己知道答案）——03 章的 faithfulness/context precision 就是补齐这条链路
3. 配置推荐（中文通用场景起步值）：chunk 512 字符 / overlap 64；BM25 $k_1=1.2$ $b=0.75$ ；RRF $k=60$ ；候选池 top-10 重排取 top-5
4. 工业栈对照：本课程手写件 $\leftrightarrow$ 生产件：分块 $\leftrightarrow$ LangChain splitter；暴力 cosine $\leftrightarrow$ FAISS/Milvus；RRF $\leftrightarrow$ Elasticsearch/Weaviate 内置；重排 $\leftrightarrow$ bge-reranker/Cohere Rerank；评测 $\leftrightarrow$ RAGAS（03 章）

练习与思考

概念检验

Q1：BM25 里参数 b 从 0 调到 1，检索行为会怎么变？

<details>

<summary> 答案</summary>

b 是文档长度归一的强度。 $b=0$ ：完全不看长度，长文档靠堆词刷分（TF 无饱和上限的旧病被 k_1 单独压制，但长文仍占优）； $b=1$ ：长度因子完全线性， $|d|$ 是平均长度两倍的文档其 TF 权重被压一半——短文档（标题、表格行）更容易浮上来。中英混合语料长度方差大时，0.75 是稳健折中；如果你的语料全是结构化短条目（FAQ），调小 b 往往更好。

</details>

Q2：RRF 为什么用 $1/(k+\text{rank})$ 而不是直接用 $1/\text{rank}$ ？

<details>

<summary> 答案</summary>

两个原因：① 稳健性—— $1/\text{rank}$ 对第 1 名 (1.0) 和第 2 名 (0.5) 差距悬殊，单榜冠军几乎垄断融合结果； $1/(k+\text{rank})$ ($k=60$) 把相邻名次的得分差压到 $\sim 1/60^2$ 量级，“多榜一致出现”比“单榜登顶”更值钱，正好符合“两路都认可 = 大概率相关”的直觉。② 抗榜单噪声——单榜的名次抖动（因打分边界毛刺引起的第 5/第 6 互换）在 k 压平后几乎不影响融合输出。极限性质（作业题 3）： $k \rightarrow \infty$ 时退化为“入选数优先、名次和次之”的计数排序。

</details>

Q3：既然 cross-encoder 更准，为什么不全程用它检索？

<details>

<summary> 答案</summary>

复杂度结构不允许。cross-encoder 必须 (query, 文档) 成对进模型： N 个文档就是 N 次前向、且每次查询都要重算（无法离线索引、无法 ANN 加速）。百万语料上一次查询 = 百万次 Transformer 前向。bi-encoder 换来了可离线、可 ANN 的结构，代价是精度。所以定式是漏斗：bi-encoder（或 BM25）从 10^6 捞 10 个 $\rightarrow$ cross-encoder 精排 10 个。这与“先用便宜的近似缩小空间、再用贵的精确计算”的 Part 13 LSH（分带粗筛 $\rightarrow$ Jaccard 精验）是同一个工程哲学。

</details>

动手实践

练习 1：体验降级路径（5 分钟）

```
`bash
```

```
RAG18_FORCE_FALLBACK=1 python courses/Part18_rag/scripts/01_minimal_rag.py
```

```
`
```

验收标准：

- [] 脚本 $rc=0$ ，打印至少 3 条 ⚠ 降级警告
- [] dense 列 recall 明显低于主模式（本机实测 0.20 vs 0.58）

- [] 能说出 hashing trick 与真嵌入的本质区别（字面碰撞 vs 语义泛化）

练习 2：加第四个查询

在脚本的 **QUERIES** 里加一条你自己的查询（先在语料快照 [data/part18_corpus/](#) 里人工确认相关 chunk 应该长什么样，再写关键词规则作为 ground truth）。

验收标准：

- [] 新查询的 4 级 recall 都有输出且能解释
- [] 观察它落在"语义型/词典型/综合型"哪一类，与本章结论对照

练习 3：chunk 大小消融

把 **CHUNK_SIZE** 改成 180 / 1024 各跑一次，记录 recall@5 变化。

验收标准：

- [] 得到"chunk 太碎丢上下文、太大稀释信号"的第一手数据
- [] 与 02 章 contextual retrieval 实验互相印证

扩展思考

- 中文 BM25 用"单字 + 二元"是通用解，但领域词典（GRPO、PagedAttention）分词后会更好——如何在引入重型分词器的前提下做领域词表？
- 检索质量的上限由分块决定、下限由重排兜底——这个说法对吗？设计实验验证。
- 如果语料每天更新 10%，哪几件套要重算？增量索引的断点在哪一层？

参考资料

- Lewis et al. 2020 《RAG for Knowledge-Intensive NLP Tasks》[arXiv 2005.11401](<https://arxiv.org/abs/2005.11401>)
- Robertson & Zaragoza 《The Probabilistic Relevance Framework: BM25 and Beyond》（BM25 权威综述）
- Cormack et al. 2009 《Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods》（RRF 原始论文）
- [Qwen3-Embedding 模型卡](<https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>)（last-token pooling 官方用法）
- [BAAI/bge-reranker-v2-m3](<https://huggingface.co/BAAI/bge-reranker-v2-m3>)（本章重排器）
- [Anthropic: Introducing Contextual Retrieval](<https://www.anthropic.com/engineering/contextual-retrieval>)（→ 02 章展开）

学完本章你能...

- [] 手写并讲清五件套每一件的角色与盲区
- [] 用 recall@k + 消融表量化每一件套的贡献
- [] 为嵌入/重排/生成配置降级路径，保证脚本永不崩
- [] 诊断"hybrid 反而变差""榜单模型水土不服"这类真实故障

[← 返回 Part 18 目录](README.md) | [下一章：02 高级 RAG →](02_advanced_rag.md)

02_advanced_rag

02 — 高级

RAG：上下文增强、结构化检索与"什么时候不该用 RAG"

- > 01 章的五件套把 recall@5 从单路 0.58/0.60 抬到混合+重排 0.92——但检索的
- > 天花板卡在一个结构性问题上：chunk 一旦切出来，就脱离了它的原文语境。
- > 本章先复刻 Anthropic 的 contextual retrieval 与 Jina 的 late chunking（贴
- > [scripts/02_contextual_retrieval.py](../scripts/02_contextual_retrieval.py) 本机
- > 实测），再鸟瞰 GraphRAG / HippoRAG 2 / RAPTOR 三个结构化思想（认知章，不实现），
- > 然后用 [scripts/03_rag_eval.py](../scripts/03_rag_eval.py) 手写 RAGAS 指标——
- > 最后回答一个更根本的问题：什么时候根本不该用 RAG。

学习目标

完成本章后，你将能够：

- 解释 chunk 失上下文问题的两派解法：contextual retrieval（先切后补）与 late chunking（先嵌后切）的本质差异
- 复现 contextual retrieval 四阶梯实验，并对照 Anthropic 官方数字解释本机量级差异（含"格式噪声与增益同量级"这一反直觉实测）
- 画出 naive → advanced → modular → agentic RAG 的演进图谱，说出 GraphRAG / HippoRAG 2 / RAPTOR 各自解决什么
- 手写 RAGAS 的 faithfulness 与 context precision，并演示评测器噪声
- 决策 RAG vs 微调 vs 长上下文三选一（带论文依据与成本量级）

前置知识

必须掌握：

- [01 章五件套](01_naive_to_hybrid.md)——本章所有实验都跑在同一条管线上

建议掌握：

- [Part 8 06 章 PPO/GRPO](../Part8_post_training/tutorial/README.md)——02 章结尾"RAG vs 微调"需要知道微调买的是什么
- [Part 14 推理部署](../Part14_inference_vllm/tutorial/README.md)——长上下文的成本结构（KV cache 随长度线性涨）是"什么时候不该用 RAG"的物理基础

可选：

- 图算法基础（/PageRank）——只影响 GraphRAG/HippoRAG 小节的阅读深度

一、上下文增强双雄

问题引入：chunk 是"断章取义"的最小单位

,

原文：《论文阅读指南》§4 逐 Part 论文实战

…… | ### Part 13 — Penedo et al. 2024 《FineWeb Datasets》 | ……

（一个 512 字符的 chunk 被切出来后，只剩孤零零的技术名词和半张表格）

查询"去重用什么算法"来的时候，这个 chunk 里的"14 band × 8 row"没有任何
"我是讲去重的"信号——语义靠语境，而语境在切块时被扔掉了。两派解法：

thropic, 2024-09)	Late Chunking (Jina AI, arXiv [2409.04701])(https://arxiv.org/abs/2409.04701)
：每个 chunk 前面拼一段 LLM 生成的"全文定位句"再嵌入	**先嵌后切：长上下文嵌入模型把**整篇文档**一次编码成 token 向量
(Anthropic 用 prompt caching 把 248M chunk 的成本压到 1.02 美元/百万 token)	零 LLM 调用；但要求嵌入模型支持长上下文 + 输出逐 token 向量
	绑定长上下文嵌入模型 (Jina 自家 jina-embeddings-v2-base-en 等)
	让上下文先于切块发生

- > 类比：contextual retrieval 像给每张从相册撕下来的照片手写备注
- > "这是 2019 年京都之行第 3 天"；late chunking 像要求看照片的人**先把整本
- > 相册翻一遍**再看单张——备注要一张张写（贵但通用），整本翻要记性好
- > （便宜但挑模型）。

实测：复刻 Anthropic 四阶梯 ([脚本 02](../scripts/02_contextual_retrieval.py))

- > 环境标注：与 01 章相同 (RTX 4090 / torch 2.6.0+cu124 / transformers 4.57.6) ；
- > 定位句由 Qwen2.5-0.5B-Instruct 贪心生成 (约 64 token，输入=文档大纲 + chunk 前 350 字) ；
- > 238 chunk 全量，脚本总耗时 50-55s (实测 52-54s，其中 LLM 定位句 ~28s；共享 GPU 上有波动)。

[Step 2] 实验一：plain → +LLM 前缀 → +BM25 混合(RRF k=60) → +rerank
query recall@20
Q1 组内相对策略梯度是哪篇论文提出的？ 0.75 0.75 0.75 0.75
Q2 SGLang 和 vLLM 各自适合什么场景？ 0.58 0.42 0.50 0.50
Q3 推理服务里 KV cache 显存碎片……怎么解 0.67 0.67 0.58 0.58

mean 0.67 0.61 0.61 0.61
失败率 0.00 0.00 0.00 0.00
(官方: 5.7% / 3.7% / 2.9% / 1.9%，累计 -67%)

[Step 3] 实验二：同样的信息、不同的前缀，recall 会怎么摆？
(章节路径 A：文档名 · 章节 原文；B：《文档名》章节：原文——信息完全相同，仅排版不同)
Q1 0.75 1.00 0.75 1.00
Q2 0.58 0.50 0.50 0.33
Q3 0.67 0.67 0.67 0.67

mean 0.67 0.72 0.64 0.67

(四列分别为：plain / +章节路径A / +章节路径B / +章节路径&LLM 句)

官方 vs 本机，逐条归因（官方数据见
[Anthropic 工程博客](https://www.anthropic.com/engineering/contextual-retrieval)，
248M chunk 语料）：

- 上下文生成质量：官方用 Claude 读整篇文档写定位句；本机 0.5B 只看大纲 + 前 350 字，定位句偶有跑题（跑脚本看 Step 1 打印的示例即可自查）——噪声前缀会把嵌入拉离查询语义。这不是实现 bug，是复刻条件的天花板
- 语料规模：官方 248M chunk 跨百万文档，"chunk 脱离文档就认不出"的问题普遍存在；本机 8 篇文档 238 个 chunk，plain 嵌入本来就不太缺上下文，

增益空间小

- 3. 评测粒度：官方指标是"top-20 一无所获"的失败率（亿级查询平均）；本机 3 个查询的失败率全为 0——指标已饱和，recall 微差纯属小样本噪声
- 4. 混合口径：+BM25 行在"带前缀文本"上算 BM25，前缀引入文档级高频词（df 被抬高），稀有词判别力被稀释；官方用加权组合 + 调参绕开
- > 本章最重要的一张表是实验二：信息一字不差、只换排版（A vs B），> mean 就从 0.72 摆到 0.64（±0.08）——**在小语料 + 通用嵌入模型上，"格式噪声" > 与"技术增益"同量级**。任何 contextual 改造必须配 A/B 评测与多样本查询，> 单点数字不可信。这正是 Anthropic 要用 248M chunk、按失败率在亿级查询上 > 平均的原因：不是炫富，是被噪声逼的。
- > △ 降级模式实测（RAG18_FORCE_FALLBACK=1）：hashing 向量 + 空定位句下 > 四阶梯 mean 仅 0.11→0.11→0.17→0.17、失败率 33%——连 top-20 都摸不到大部分相关 > chunk。上下文工程救不了烂嵌入。

工程启示（按性价比排序）：

- 1. 先上确定性结构前缀（文档名/章节路径/元数据）——零成本、方向对、可复现
- 2. LLM 定位句是"语料大、生成模型强、有 prompt caching 摊成本"时才划算的选项
- 3. 上下文工程的收益来自信息量，不是"加前缀"这个动作本身

Late Chunking 的展开（选读）

传统: 文档 → 切块 → 逐块嵌入 每块独立编码，语境归零
Late: 文档 → 整篇编码(token 级向量序列) → 按边界切 → 逐块 mean-pool
第 i 块的每个 token 向量都"看过"全文，
池化出来的块向量自带长程语境

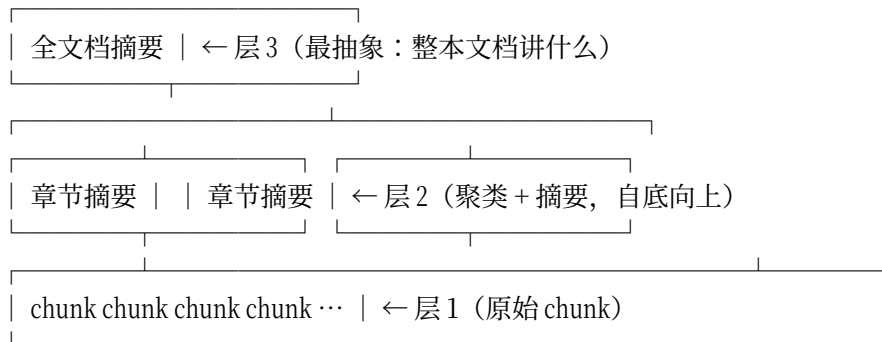
Jina 的实测（[论文 2409.04701](https://arxiv.org/abs/2409.04701) / [博客](https://jina.ai/news/late-chunking-in-long-context-embedding-models/)）：
在长文档检索基准上，late chunking 显著优于传统"先切后嵌"，且不需要任何 LLM 调用。代价是嵌入模型必须支持长上下文并暴露 token 级输出——Qwen3-Embedding-0.6B 的 32k 上下文理论上可行（本课程留作扩展思考，未实现）。

二、结构化检索思想（认知小节，不实现）

RAG 的演进不是零件替换，是检索结构的代际跃迁（综述见 arXiv [2501.09136](https://arxiv.org/abs/2501.09136) Agentic RAG Survey）：

代际	代表	检索结构	解决什么
Naive RAG	Lewis 2020	平面 chunk 列表	有没有资料可查
Advanced RAG	混合/重排/contextual（01 章 + 本章 § 一）	平面 + 查询改写 + 上下文增强	查得准不准
Modular RAG	RAPTOR / GraphRAG / HippoRAG 2	**树 / 图**	多跳问题、全局性问题
Agentic RAG	Search-R1、ReAct 式（→ Part 19）	检索成为**工具**，模型决定查几轮查什么	检索策略本身

RAPTOR：把语料组织成一棵"摘要树"（arXiv [2401.18059](https://arxiv.org/abs/2401.18059)）



- 痛点：细节问题要底层 chunk, "这本书的主线论点是什么"要上层概括——平面检索只能命中其一
- 做法：对 chunk 做向量聚类 → 每簇生成摘要 → 摘要再聚类再摘要, 形成树；检索时可在多层上并行取证据
- 一句话：把"局部细节"和"全局要义"放进同一棵可检索的树里

GraphRAG：先建知识图谱，再检索 (arXiv [2404.16130](https://arxiv.org/abs/2404.16130))

原文 --LLM 抽取--> 实体/关系三元组 --聚类--> 社区(community) --逐社区摘要--> 图摘要
 查询来了：
 局部查询 → 图邻域扩展（种子实体的多跳邻居）
 全局查询 → 遍历社区摘要（"整个语料对 X 的态度"这类问题）

- 痛点：平面 RAG 对"全局性/多跳"问题天然残废（答案分散在几百个 chunk 里, 没有单块能命中）
- 代价：建图阶段的 LLM 调用成本高（微软原版对语料做多轮实体抽取）
- 一句话：把检索从"相似度匹配"升级成"图上的推理"

HippoRAG 2：检索即记忆 (arXiv [2502.14802](https://arxiv.org/abs/2502.14802))

离线: chunk → LLM 抽取三元组 → 知识图谱(节点=实体) + Personalized PageRank 备好
 在线: query → 抽实体作为种子 → 图上跑 Personalized PageRank → 激活的节点带回 chunk

- 思想：模仿海马体记忆索引——用图上的随机游走做"联想", 一次查询能带出多跳之外的知识；参数化的 LLM 负责读, 非参数化的图负责记 ("From RAG to Memory")
- 一句话：把"非参数化持续学习"做成图上的联想检索

> 三者共同的底层判断：**当问题不再是一段话能回答的时候，检索结构本身要升级**。
 > 树（RAPTOR）管抽象层级，图（GraphRAG/HippoRAG 2）管多跳关联。
 > 工程取舍：结构化索引的构建成本（LLM 调用）vs 平面索引的召回上限——
 > 私有语料小、问题多跳多时才得上结构。

三、什么时候不该用 RAG

> △ 这一节在面试里的价值不亚于"会用 RAG"——说得出边界才证明真懂。

1. LaRA (arXiv [2502.09977](https://arxiv.org/abs/2502.09977))：没有银弹。
在多任务、多模型维度上系统对比 RAG 与长上下文 LLM，结论是两者各有胜负域，不存在全面占优的一方——选型必须回到任务分布。

**2. Self-Route (arXiv [2407.16833](https://arxiv.org/abs/2407.16833), "Retrieval Augmented Generation or Long-Context LLMs?")：成本
差数倍，让模型自己路由。**
论文实测：长上下文 LLM 平均效果更好，但 RAG(k=5) 的 token 消耗只约为长上下文直塞的 17% (约 1/6——长 prompt 的 KV cache 线性膨胀，→ [Part 14](../Part14_inference_vllm/tutorial/README.md))；
提出的 Self-Route 让模型先判断"这题需要全文吗"，只把真正需要长上下文的查询路由给全文模式——效果接近纯长上下文，token 成本比纯长上下文省 39%~65% (Self-Route 的 token 占长上下文的 38.6%~61%，仍高于纯 RAG 的 17%)。

3. Context engineering 共识：装得下就别绕路。
当上下文预算（现代模型 128k-1M token）轻松装下全部相关知识（<200k token 的私有文档、一两次会话的记忆），直接把材料塞进 prompt 是更简单、更可靠、更易调试的方案——RAG 引入的每个组件（分块/嵌入/检索/重排）都是新的误差源与运维面。RAG 的真正战场是：知识量超出上下文预算、知识高频更新（重嵌入比重训便宜亿万倍）、需要引用出处（可审计性）。

一张决策表：

	首选	理由
直接塞 prompt		零检索误差、
用 RAG		重嵌入 << 重训
微调 (→ [Part 8](../Part8_post_training/tutorial/README.md)、[Part 12](../Part12_finetune_llamafactory/tutorial/README.md))		RAG 改不了行
微调 + RAG 叠加		两者正交：参
模块化/Agentic RAG		平面检索召回

> 类比：长上下文 = 把整本百科全书搬进考场（贵但全）；RAG = 考场配图书管理员
> （便宜但要赌他找得对）；微调 = 让学生变成领域专家（最贵但改变的是人不是书）。

四、RAGAS：让"答案质量"可测量

01 章止步于 recall@k（检索指标）——但用户感知的是答案。RAGAS（业界最常用的 RAG 评测框架）用四个 LLM-as-judge 指标补上这条链路：

指标	定义	判什么
Faithfulness（忠实度）	answer 拆成原子 claims，逐条判"是否被 contexts 支持"；分数 = 支持/总数	幻觉（答案有没有编）
Answer Relevancy（答案相关性）	从 answer 反向生成问题，与原 query 算相似度	跑题（答非所问）
Context Precision（上下文精确率）	检索回的 contexts 逐条判"对回答有用吗"，按 AP 口径聚合	噪声（检索塞没塞无关材料）
Context Recall（上下文召回率）	ground truth answer 逐句判"能否在 contexts 里找到依据"	漏检（该查的查到没有）

实测：手写 faithfulness / context precision ([脚本 03](../scripts/03_rag_eval.py))

- > 环境标注：同前；裁判 = Qwen2.5-0.5B-Instruct 贪心解码 (max 8 token) ；
- > 评测对象 = 01 章同款管线 (hybrid+rerank top-5 证据) 的生成答案；
- > "幻觉版" = 在正确答案后拼两句无中生有的话。总耗时实测 17s。

[Step 2] faithfulness：grounded vs 拼接幻觉句

Q1: claims 6→8 条 | grounded=0.67 | +幻觉=0.50

Q2: claims 1→3 条 | grounded=0.00 | +幻觉=0.00

Q3: claims 5→7 条 | grounded=1.00 | +幻觉=0.71

mean: grounded=0.56, +幻觉=0.40 (幻觉句拉低 0.15——若没拉低, 说明裁判太弱)

[Step 3] context precision (top-5, AP 口径) + 裁判相关性 vs 检索排名

Q1: judge 逐位判定 [1, 1, 1, 1, 0] | context_precision=1.00 | Kendall τ =+1.00

Q2: judge 逐位判定 [1, 1, 1, 1, 1] | context_precision=1.00 | τ =n/a (标签无区分度)

Q3: judge 逐位判定 [1, 1, 1, 1, 1] | context_precision=1.00 | τ =n/a

[Step 4] 评测器噪声：固定同一输入、只换 prompt 措辞

幻觉句 (期望 no) 三种 entailment 问法: A=no / B=yes / C=yes → $\triangle$ 翻转

相关 chunk (期望 yes) 三种相关性问法: 「相关吗」=yes / 「有用吗」=no / few-shot=no

三条读数：

1. faithfulness 有判别力但不完美：幻觉句把分数从 0.56 压到 0.40——方向对、幅度被裁判能力封顶。Q2 的 grounded=0.00 是"裁判误杀" (正确短答案"手写代码清单叫 TOP8"被判不支持)：0.5B 裁判的绝对分数不可信
2. context precision 的措辞陷阱：问"有用吗"时 0.5B 几乎一律答 no (全 0 标签)，换成"相关吗"立刻恢复正常——裁判 prompt 本身是最大的超参数
3. 评测器噪声是结构性的：同一陈述、等价问法，判决在 yes/no 之间翻转；few-shot 示例反而把 0.5B 带偏。工程对策：固定 prompt 模板 + 多次采样投票 + 只做系统间相对比较 (A/B 谁高谁低可信，绝对值 0.56 vs 0.72 不可信)

- > ragas 本身在本环境未安装——脚本 03 的处理方式就是教程要教的姿势：
- > import 失败 → 打印 `uv pip install --python .venv ragas` 指引 → 跳过该段，
- > rc=0。手写指标与 RAGAS 同构 (claims 拆解 + 逐条 entailment)，装不装框架
- > 都能跑通同一条评测链路。

练习与思考

概念检验

Q1：contextual retrieval 和 late chunking 都在解决"chunk 失语境"，为什么工程界先大规模落地了前者？

<details>

<summary> 答案</summary>

因为兼容性。contextual retrieval 只是"改了送进嵌入模型的文本"，下游

(嵌入模型/向量库/检索/重排) 零改动，任何存量系统加一层 LLM 前缀生成就能上线；

late chunking 要求嵌入模型本身支持长上下文并暴露 token 级输出——这把技术选型绑死在特定模型家族上。工程里"在哪一层打补丁"往往比"哪个补丁更优雅"

更决定落地速度。（成本上 contextual retrieval 有了 prompt caching 之后也不再是障碍——Anthropic 把 248M chunk 的上下文生成成本做到了约 1.02 美元/百万文档 token。）

</details>

Q2：RAGAS 四个指标里，哪两个可以不用 LLM 裁判？怎么做？

<details>

<summary> 答案</summary>

严格说四个都可以换实现，但最自然"去 LLM 化"的是 context precision 和 context recall：有标注的相关性标签（01 章的关键词规则 ground truth 就是一种）时，context precision = 检索列表前 k 位中相关的比例（AP 口径手算），context recall = ground truth 证据被检索列表覆盖的比例——纯确定性计算。而 faithfulness / answer relevancy 本质是语义判断（"这句话算不算被支持"），规则替代的误差大（03 章降级模式的关键词裁判就抓不出幻觉句，实测 grounded=1.00、+幻觉=1.00——降级运行日志留档可复现）。所以生产上常见组合：离线回归用确定性指标，抽样审计用 LLM 裁判。

</details>

Q3：老板说"把产品手册全量塞进 2M 上下文的模型，撤掉 RAG 省事"——你会怎么回应？

<details>

<summary> 畅所欲答版</summary>

分四步算账：① 延迟与成本：长 prompt 的 KV cache 随长度线性涨，Self-Route (2407.16833) 实测 RAG 的 token 消耗仅约为长上下文的 1/6，且长上下文每次提问都要付全量 token 钱；② 效果的迷失：LaRA (2502.09977) 显示长上下文并非全面占优；超长上下文还存在"lost in the middle"现象（中间段信息利用率下降）；③ 更新频率：手册改一版就要重发全量 prompt（或重造缓存），RAG 只需重嵌入改动的 chunk；④ 可审计性：产品场景常要"这句话出处是哪页"，RAG 天然带引用。合理方案是 Self-Route 式路由：简单查询走检索，确需全局比对时才放长上下文。

</details>

动手实践

练习 1：量化你的裁判

给 [脚本 03](../scripts/03_rag_eval.py) 的 **judge** 写一个"全 yes 裁判"

(**lambda p: 'yes'**) 和一个"诚实裁判"（按关键词重合度），对比 faithfulness 分数差异。

验收标准：

- [] 全 yes 裁判下幻觉版与 grounded 版分数相同（=裁判无判别力）
- [] 能解释为什么"裁判的绝对分数要校准、相对比较才可信"

练习 2：late chunking 思想最小复现（进阶）

不用长上下文模型，用"整篇文档嵌入 + chunk 与文档向量的凸组合"近似"chunk 自带全文语境"，测 recall@20 相比 plain 的变化。

验收标准：

- [] 与 02 章实验二的结构化前缀对照（同为"给块加全局信息"的廉价近似）
- [] 得出你自己的结论：凸组合权重 α 的敏感性如何

练习3：跑一遍 Anthropic 官方博客的数字

读 [Introducing Contextual Retrieval](https://www.anthropic.com/engineering/contextual-retrieval), 把官方实验设置（语料、指标、各阶段数字）整理成表，与脚本 02 输出逐行对照，列出每个“不可比因素”。

验收标准：

- [] 表格覆盖：语料规模 / 指标口径 / 上下文生成模型 / 融合方式
- [] 用自己的话解释“为什么复刻方向正确 ≠ 复现幅度”

扩展思考

- HippoRAG 2 的 Personalized PageRank 与 Part 13 的 LSH 都在“用随机化换可算性”——这个哲学还能不能在 RAG 里找到第三处应用？
- Agentic RAG（→ Part 19）把“查不查、查什么、查几轮”交给模型决策——这会把评测从“单次检索质量”变成什么形态？
- RAGAS 的裁判换成 70B 模型，评测器噪声会消失吗？设计实验回答。

参考资源

- Anthropic 《Introducing Contextual Retrieval》[工程博客](https://www.anthropic.com/engineering/contextual-retrieval)
- Late Chunking: Long-Context Embedding Models (arXiv [2409.04701](https://arxiv.org/abs/2409.04701)) / [Jina 博客](https://jina.ai/news/late-chunking-in-long-context-embedding-models/)
- Agentic RAG 综述 (arXiv [2501.09136](https://arxiv.org/abs/2501.09136))
- GraphRAG: From Local to Global (arXiv [2404.16130](https://arxiv.org/abs/2404.16130)) · HippoRAG 2: From RAG to Memory (arXiv [2502.14802](https://arxiv.org/abs/2502.14802)) · RAPTOR (arXiv [2401.18059](https://arxiv.org/abs/2401.18059))
- LaRA 基准 (arXiv [2502.09977](https://arxiv.org/abs/2502.09977)) · Self-Route (arXiv [2407.16833](https://arxiv.org/abs/2407.16833))
- MTEB 维护性研究 (arXiv [2506.21182](https://arxiv.org/abs/2506.21182)) · [RTEB](https://github.com/NovaSearch-Team/RTEB)
- [RAGAS 官方仓库](https://github.com/explodinggradients/ragas)

学完本章你能...

- [] 说清 contextual retrieval / late chunking / 结构化前缀各自的位置
- [] 用“格式噪声与增益同量级”的实测结论解释为什么评测要多样本
- [] 画出 naive→advanced→modular→agentic 演进图并给出选型判据
- [] 手写 faithfulness / context precision 并校准裁判噪声
- [] 面对任何需求先回答“该不该用 RAG”

[← 上一章：01 手写五件套](01_naive_to_hybrid.md) | [返回 Part 18 目录](README.md)

- > 下一站 Part 19 (Agent)：检索从“管线的一个阶段”变成“模型的一个工具”——
- > 模型自己决定什么时候查、查什么、查完够不够，Agentic RAG 把本章的检索结构
- > 交给策略来学（与 [Part 17 Agentic RL](../Part17_agentic_rl/tutorial/README.md)
- > 的多轮轨迹训练衔接）。